LiveRecall: Revision-Aware Caption Delivery and Recovery

CS3103 Assignment 4 | Team 13

1. Objective and architecture

LiveRecall distributes a changing caption stream to concurrent browser clients while retaining enough recent context for students who miss part of a lecture. Our implementation separates the deterministic caption source, an asynchronous Python WebSocket server, and a browser interface. It supports live revision replacement, local 15/30/60-second recall, automatic reconnect, incremental recovery, late join, and bounded per-client delivery queues. Speech recognition is outside the mandatory implementation.

The data path is:

```
caption_source.py -> solution_server.py: validate revision
                     |-- final: save to HistoryStore
                     `-- queue in each client's outbox -> browser
```

`solution_server.py` accepts `--port` and uses `asyncio` and `websockets`. `CaptionRevisionState` validates incoming revisions; `HistoryStore` retains recent finals; and `ClientOutbox` separates source ingestion from each client's socket writes. The browser stores caption objects and their displayed rows in a Map. Keeping these responsibilities separate allows recovery history to expire without losing the browser's finalized resume position.

2. Protocol and caption-state correctness

The source identifies the lecture with `SOURCE_HELLO`. Each `CAPTION_UPDATE` contains `session`, `segment`, `revision`, `source_ts`, `final`, and `text`. We preserve the original accepted message when forwarding it, including extra fields. Mandatory operation uses one active source and session `L1`.

The server stores one latest accepted caption for revision validation. This uses the supplied source's sequential behavior: it completes one segment before starting the next. For the same segment, only a strictly higher revision can replace a provisional caption. Once final, that segment cannot be overwritten. An earlier segment is rejected; a later segment starts its own revision counter. Positive integer segment/revision fields, a Boolean final flag, and the source session are checked. The implementation does not assume three revisions per segment, although the public generator produces three.

The browser keeps one Map entry per segment, containing its latest accepted message and DOM row. An accepted revision updates the existing row rather than appending a duplicate. Duplicate or lower revisions and updates to an existing final are ignored. Provisional and final captions have explicit labels. A fixed-height scrollable live area keeps the recall controls in place. Caption text is rendered with `textContent`.

For recall, the browser retains finalized captions for 70 seconds using `source_ts`; cleanup runs on updates and once per second while idle. The "What did I miss?" button selects the preceding 30 seconds, with 15- and 60-second alternatives. Recall filters locally stored finals and orders them by segment; clicking a recall button does not request server history.

3. Bounded history, reconnect and late join

The server's history dictionary is keyed by `(session, segment)` and stores one finalized caption per key. Entries older than 70 seconds are removed on history insertion or query. This provides a bounded time window with a margin beyond the maximum 60-second recovery request. The revision tracker remains constant in size; it does not retain another lecture transcript.

On connection, the client sends `CLIENT_HELLO` with its latest finalized `last_segment`/`last_revision` and requested `history_seconds`. The browser requests 60 seconds. New clients use `(0, 0)`. The server responds with `HISTORY` containing only recent finals strictly after the reported position, ordered oldest first. The response is incremental rather than a replay of the lecture.

After a socket closes, the browser schedules one reconnect attempt after one second. Another failed connection can schedule another attempt. Connection status is displayed, and guards prevent old sockets or duplicate timers from starting overlapping retries. Only accepted finals advance the resume position; provisionals do not. This position survives local history expiry within the open page, while a page reload starts a fresh client state.

History items pass through the same browser acceptance function as live updates. Recovered finals can replace missed provisional states without creating another row. New recovered rows are inserted in segment order. On the server, history selection, queuing the HISTORY response, and registering the client occur without an intervening `await`. A single sender then transmits HISTORY before subsequent live updates. This avoids the missed-update gap that would arise from awaiting history delivery before subscribing the client.

4. Slow-client isolation and overload policy

The earlier broadcaster awaited sends to all clients before reading further source updates. Our final design instead puts each accepted caption into every client's outbox without awaiting a socket. Each outbox has one independent sender task. A blocked sender therefore does not directly block source ingestion or another client's sender, although all tasks still share the event loop and CPU.

An outbox uses a deque. When a newer update for a segment arrives, it removes older queued provisional updates of that segment and appends the new update. Queued finals and HISTORY are protected from this coalescing. Once sent, messages cannot be replaced. When the sender keeps up, each revision leaves the queue before its successor arrives, allowing full update coverage.

The queue accepts at most 32 pending messages before declaring overflow. On each insertion it also checks whether the oldest remaining queued message is more than 15 seconds old. Overflow marks the queue unusable, clears its pending messages, and wakes its sender. When the sender next obtains the overflow signal, it closes the connection with code 1013; the browser can reconnect and request missed finals from server history.

This distinction matters: finals are protected during coalescing, but overflow does clear queued finals. Recovery depends on reconnecting while those finals remain in the requested history window. The age check is insertion-driven, not an independent timer, and a sender blocked inside `ws.send()` cannot react to the overflow signal immediately. Consequently, the policy bounds the application queue but does not guarantee delivery or disconnection within 15 seconds.

We configure a 4 KiB transport write limit and a 16 KiB socket send-buffer request to encourage backlog to remain in the application outbox, where it can be coalesced. These settings do not cap every buffer across TCP and the receiver; in particular, they cannot remove messages already buffered at the client.

5. Experimental method and public results

The integrated T1-T5 runs used `develop` at `54b733a` on native Ubuntu 24.04.4, an Intel i7-13620H host with 16 threads and 15 GiB RAM, Python 3.12.3, and websockets 16.1.1. Server and simulated clients ran on the same host over loopback. Each workload ran three independent times with a fresh server, seed 3103, and a 500 ms freshness threshold. The supplied measurement programs were not modified. Earlier VM results are retained in Git history but are not used as a controlled comparison against this different machine.

Each reported number is the median of the three per-run metrics. We do not pool latency samples across runs before computing percentiles. Saved JSON summaries for all 24 runs used below match re-analysis of their CSV logs. Commands and individual run links are in `README.md` and `EXPERIMENT_RESULTS.md`.

Caption Age is client receive time minus the source timestamp. P50/P95/P99 and Fresh Caption Ratio describe accepted live updates, excluding recovered history items. Final completeness counts unique finals received live or through history against generated finals for all clients. Fresh Final Caption Ratio also uses generated finals as its denominator. Normal Update Coverage counts unique live revisions against generated revisions, so discarding provisionals cannot be hidden behind good final-only results.

HISTORY recovery time measures receipt of the recovery response after connection, not the preceding offline period or browser rendering time.

T1: Basic correctness

With 20 clients for 20 seconds, each run generated 22 finals and 66 revisions. Median P50/P95/P99 Caption Age was **2.88 / 4.36 / 5.24 ms**. Fresh Caption Ratio, Fresh Final Caption Ratio, final completeness, and update coverage were all **100%**. Every run recorded zero duplicates, stale updates, invalid non-final history items, and connection errors. All 1,320 expected live revisions and 440 final deliveries were received per run. This supports normal delivery correctness, while optional regression tests cover deliberately stale and duplicate inputs that the normal source does not generate.

T2: Public scale

With 100 clients for 40 seconds, each run generated 49 finals and 147 revisions. Median P50/P95/P99 was **6.76 / 12.08 / 15.25 ms**. Both freshness ratios, completeness, and coverage remained **100%**, with zero duplicate/stale updates or connection errors. Each run delivered 14,700 live revisions and 4,900 finals. Latency increased relative to T1, consistently with more delivery and logging work, but client count is not isolated from duration and scheduling effects. These results demonstrate the tested 100-client capacity, not unlimited scale.

T3: Slow readers

Each 60-second run used 100 clients, including 10 that slept 500 ms after each message. We tested the naive broadcaster (`1532e6a`) and outbox implementation on the same host and workload.

Metric: median of three runs	Naive	Outbox
P50 Caption Age (ms)	6.66	7.12
P95 Caption Age (ms)	2254.08	2253.62
P99 Caption Age (ms)	24017.88	24020.49
Fresh Caption Ratio	94.65%	94.65%
Fresh Final Caption Ratio	90.00%	90.00%
Final completeness	95.13%	95.13%
Update coverage	95.18%	95.18%
Normal-client P95 (ms)	9.82	11.02

Normal readers remained fresh, but slow readers' P95 reached about 28 seconds. The two designs produced essentially the same results: the outbox server logged zero coalesced messages and zero overflow disconnects in all three runs. Compressed caption frames were small enough for client-side receive buffers to absorb the backlog, so server writes did not encounter backpressure. Slow clients had not read every final by the measurement cutoff. Their missing deliveries still count against the measured 95.13% completeness; the result does not establish complete slow-client delivery.

One additional T3 memory observation sampled the server for 70 seconds: resident memory was approximately 26.3 MB at startup, 31.6 MB after clients connected, and 32.5 MB at the end. This is evidence of modest memory use over that run, not proof of bounded behavior for arbitrary runtimes or input rates. T3 alone does not validate coalescing, motivating the bandwidth experiment below.

T4: Reconnect

Each 60-second run used 100 clients. Twenty disconnected at 15 seconds, remained offline for five seconds, and reconnected. Median P50/P95/P99 was **6.30 / 10.19 / 12.87 ms**. Completeness was **100%**, live freshness **100%**, fresh-final ratio **98.42%**, and update coverage **98.25%**. Every run had 120 successful connections and zero duplicates, stale updates, invalid history items, or connection errors. The runs recovered 140, 140, and 139 finals.

Median HISTORY response P50/P95 was **2.46 / 3.68 ms**. Combined with complete final delivery and zero duplicates, this supports incremental recovery under the tested outage. Some recovered finals were older than 500 ms, reducing the fresh-final ratio. Provisionals missed while offline were not replayed, reducing coverage. This is the intended distinction between recoverable final state and transient revisions.

T5: Network impairment

Each 40-second run used 100 clients with loopback `tc/netem` applying 120 ms delay, 30 ms jitter, and 3% packet loss. Median P50/P95/P99 was **252.55 / 632.29 / 911.35 ms**. Live freshness was **93.39%**, fresh-final ratio **90.61%**, completeness **100%**, and coverage **97.85%**. All runs had zero duplicate/stale updates, invalid history items, and connection errors.

The increased delay and latency tail are consistent with added propagation delay, jitter, and TCP retransmissions. WebSocket's ordered TCP stream means loss can delay subsequent data even though it is eventually delivered. Coverage ranged from 92.45% to 99.26%; queued provisional replacement can reduce coverage when a connection backs up. Initial history also recovered a few finals for clients whose handshakes completed after the source began. These observations should not be interpreted as missing finalized state: every client received all 49 finals in each run.

6. Before/after optimization under bandwidth overload

To exercise server-side backlog, we compared the naive and outbox servers on the same host with 600 clients for 40 seconds, three runs each. The supplied profile limited loopback to 2 Mbit/s with a 400 ms token-bucket queue setting. The final server code matches the integrated version used above.

Metric: median of three runs	Naive	Outbox
P50 Caption Age (ms)	1228.30	1170.41
P95 Caption Age (ms)	4583.83	2851.54
P99 Caption Age (ms)	9842.87	6182.79
Fresh Caption Ratio	0.69%	0.52%
Fresh Final Caption Ratio	0.64%	0.00%
Final completeness	97.85%	99.62%
Update coverage	92.74%	74.56%

P95 improved by approximately **38%** and P99 by **37%**, while completeness rose by **1.77 percentage points**. Every outbox run beat every naive run on P95 and completeness; outbox completeness ranged from 99.16% to 100%. The outbox server coalesced 49,618 provisional updates across three runs and recorded no overflow disconnects. Coalescing saved bandwidth for newer state and finals, at the cost of intermediate revisions: coverage fell by 18.18 percentage points.

This is a latency/completeness improvement, not a freshness improvement. Freshness stayed near zero, and fresh-final ratio decreased to zero. The link remained overloaded and messages already in transport/receiver buffers could not be replaced. Final delivery also remained imperfect in two outbox runs. An earlier 1,000-client bandwidth attempt was unsuitable for comparison because handshake timeouts dominated; reducing to 600 clients allowed useful comparison of connected-client delivery. This differs from an unconstrained 1,000-client scale test.

7. Limitations and future work

The public tests show complete normal and short-outage final delivery, but not complete delivery in every stressed workload. T3's slow clients remained stale, and the bandwidth experiment retained significant tail delay. Queue length is bounded in messages, not bytes; time-based history size also depends on source rate. The revision tracker assumes sequential segments, and simultaneous active segments would require different state management. Recovery is limited to recent available finals, so a sufficiently long outage or delayed overflow close can exceed its window. A 15-second queue-age setting alone does not guarantee that the client's actual resume position is within that window.

The measurements use one machine and the public deterministic source. They do not establish results for hidden burst patterns, larger populations, or independent client networks. Load clients measure networking behavior rather than DOM rendering or user comprehension. Optional regression tests cover revision rules, history expiry, recovery merges, queue replacement, and a deliberately stalled connection; they supplement the required experiments. The separate browser demonstration must verify the visible workflow with at least two clients, recall, reconnect recovery, and late join.

Future work should add application-level progress acknowledgements to distinguish socket delivery from caption consumption, and evaluate byte-based buffering and an independent send/lag watchdog. Experiments should measure convergence after a bottleneck is removed, including whether all missed finals remain recoverable. As an FYP direction, a classroom deployment could combine this delivery protocol with a separate local ASR source and study recall usability under real lecture conditions, while measuring caption-generation delay separately from networking and rendering delay.

References and supporting material

1. CS3103 Assignment 4, *MiniProject LiveRecall*, assignment specification, updated 17 October 2026.

2. Supplied `caption_source.py`, `run_demo.py`, `load_clients.py`, and `analyse_results.py`: workload generation and metric definitions.

3. Team 13 repository: `EXPERIMENT_RESULTS.md` and associated `{t1,t2,t3,t4,t5,t3before,bw600_before,bw600_after}_run01` to `_run03` CSV/JSON files; `t3_memory.csv` contains the additional memory observation.

4. `README.md` provides reproduction commands; `DEMO_GUIDE.md` describes browser verification; `AI_USAGE.md` and `CONTRIBUTIONS.md` declare assistance and work allocation.